

Logbuch — KI1-Projekt 308

Name: Kolja Scriba

Gruppe: 308

Zeitraum: 20.02.2026 – 15.04.2026

Eintrag 1

Datum: 20.02.2026 10:00-12:00

Titel: Projektstart - Besprechen von Flos Arbeit und Einrichten der Umgebung

Aus Gruppentreffen: Gruppentreffen 3 (20.02.2026)

Arbeitsschritte

- Gruppentreffen mit Flo um Projekt erklärt zu bekommen
- Probleme beim Start auf meinem Rechner da Python 3.14.3 nicht mit Tensorflow 2.18 funktioniert da ich einen Intel-Mac habe

Schwierigkeiten:

- Verständnis für Architektur und aufbau von IOS mit VSS Code auf Intelbasierten Macs

Eintrag 2

Datum: 21.02.2026 13:00-15:30

Titel: Einrichtung der Python-Entwicklungsumgebung

Arbeitsschritte:

- Ich habe Python 3.13 über das Terminal mit Homebrew installiert.
- Beim Erstellen einer virtuellen Umgebung gab es zunächst ein Problem wegen eines unzulässigen Zeichens im Ordnerpfad.
- Beim Installieren von TensorFlow trat ein Fehler auf, weil TensorFlow Python 3.13 nicht unterstützt.
- Zusätzlich habe ich gelernt, dass auf meinem Intel Mac maximal TensorFlow 2.16.2 unterstützt wird.
- Deshalb bin ich auf Python 3.11 umgestiegen, da diese Version stabil mit TensorFlow funktioniert.

- Ich habe verstanden, dass das Terminal und das Jupyter-Notebook unterschiedliche Python-Interpreter verwenden können.
- Der Fehler „ModuleNotFoundError: No module named 'numpy'“ entstand, weil das Notebook nicht die richtige virtuelle Umgebung nutzte.
- Ich habe gelernt, wie man in VS Code den richtigen Kernel auswählt und die virtuelle Umgebung korrekt verbindet.
- Am Ende habe ich mein Setup so angepasst, dass Python-Version, virtuelle Umgebung und Notebook-Kernel übereinstimmen.

Entscheidungen:

- Ich habe Python 3.11.14 installiert und werde forthin damit Arbeiten, da keine neuere Version möglich ist

Schwierigkeiten:

- Es war zunächst unklar, welche TensorFlow-Version auf meinem Intel Mac überhaupt verfügbar ist
- Ich hatte Probleme beim Erstellen der virtuellen Umgebung wegen eines ungültigen Zeichens im Ordnerpfad
- Es war verwirrend, dass Terminal und Jupyter-Notebook unterschiedliche Python-Interpreter verwenden können
- Der Fehler „ModuleNotFoundError“ war schwer einzuordnen, weil die Pakete eigentlich installiert waren
- Die Auswahl des richtigen Kernels in VS Code war anfangs unübersichtlich.
- Mehrere installierte Python-Versionen auf meinem System haben zusätzlich für Verwirrung gesorgt
- Die Fehlermeldungen von pip waren technisch formuliert und nicht sofort eindeutig verständlich

Eintrag 3

Datum: 21.02.2026 15:30-18:00

Titel: nach Kompatibilitätstest commiten jedoch musste dafür venv geändert werden und Lasso Ridge um Polygrad 3 erweitert

Arbeitsschritte:

- ausführen aller unterschiedlichen Aufgaben um Kompatibilität mit python 3.11.14 zu gewährleisten
- Ich habe mein aktuelles virtuelles Environment (.venv) aktiviert.
- Ich habe nbstripout im aktiven Environment neu bzw. aktualisiert installiert.
- Ich habe nbstripout --install ausgeführt, damit der Git-Filter neu gesetzt wird.
- Dadurch wurde der alte, ungültige Python-Pfad in der Git-Konfiguration durch den korrekten Pfad

meines aktuellen Environments ersetzt.

- Anschließend konnte ich wieder normal git add und git commit ausführen.

Lasso/Ridge um PolyGrad 3 erweitert

Schwierigkeiten / offene Fragen:

- Git hatte auf eine nicht mehr existierende Python-Datei (.venv/bin/python3.13) verwiesen.
- Durch die Neuinstallation wurde der Filter mit dem richtigen Interpreter verknüpft.
- Der Fehler clean filter 'nbstripout' failed trat danach nicht mehr auf.
- Danach konnte ich alles wieder commien

Eintrag 4

Datum: 25.02.2026 10:00 - 18:00

Titel: Neural Network Logbuch Early Stop eingefügt um Overfitting beim NN zu vermeiden

Aus Gruppentreffen: Eigenständig

Arbeitsschritte:

- lokaler Main Branch und origin main Branch auf Git Hub haben unterschiedliche Commits
- Lösung dafür git pull --rebase origin main sorgt dafür dass die lokalen Commits oben auf die Remote Änderungen draufgesetzt werden
- versuchen eines Early Stopping
- für die Implementierung wurde eine Copy von Notebook 5 erstellt
- Early Stopping in der Funktion train_and_evaluate() implementiert
- Keras-Callback EarlyStopping verwendet
- Überwachung des Validierungsfehlers (monitor="val_loss")
- patience=20 gesetzt → Training stoppt nach 20 Epochen ohne Verbesserung
- restore_best_weights=True aktiviert → beste Gewichte werden automatisch wiederhergestellt
- Callback in model.fit() über callbacks=[early_stop] eingebunden
- Ziel: Overfitting vermeiden und Trainingszeit reduzieren
- Beobachtung: Bei 100 Epochen kein vorzeitiger Abbruch → Modell zeigte noch Verbesserung
- Maximale Epochenzahl auf 300 erhöht
- Training stoppte automatisch bei Epoche 74
- Overfitting wurde dadurch reduziert
- Trainingszeit wurde effizient genutzt

Entscheidungen:

Schwierigkeiten / offene Fragen:

- Verständnis warum Main Breach und Origin Breach unterschiedliche Commits haben

Eintrag 5

Datum: 26.02.2026 10:00-16.00

Titel: l2 regularisierung und vergleich der verschiedene

Aus Gruppentreffen: Eigenständig

Arbeitsschritte:

- Unterschiedliche Regularisierungsverfahren integriert (L2, Dropout, L2 + Dropout)
- Overfitting-Gap berechnet und Modelle systematisch verglichen
- Vergleichstabelle mit R^2 Train, R^2 Test, MAE Test, Epochen und Parameteranzahl erstellt
- Ergebnisse interpretiert und strukturell überarbeitet
- Struktur des Notebooks reflektiert

Entscheidungen:

Schwierigkeiten / offene Fragen:

Eintrag 6

Datum: 06.03.2026 13:00-18:00)

Titel: Neuronales Netz verbessern

Aus Gruppentreffen: Treffen 4

Arbeitsschritte:

- Projektstruktur im Repository angelegt und verschiedene Notebook-Dateien erstellt (EDA, Baseline, Scaling, NN).
- Datensatz California Housing Dataset geladen.
- Zielvariable (y) und Feature-Matrix (X) definiert.

- Datenvorbereitung über eigene Funktionen (load_and_clean_data, get_train_test_split) durchgeführt.
- Trainings- und Testdaten erstellt (x_train, x_test, y_train, y_test).
- Features skaliert (Standardisierung), um sie für neuronale Netze geeignet zu machen.
- Validierungsdatensatz aus den Trainingsdaten erstellt.
- Dimensionen der Trainings-, Validierungs- und Testdaten überprüft.
- TensorFlow/Keras in der virtuellen Umgebung eingebunden.
- Imports für:
- TensorFlow
- Keras
- Layers
- EarlyStopping
- Pandas
- Matplotlib eingerichtet.
- Eigene Modellfunktion build_model_regularized erstellt.
- Trainingsfunktion train_and_evaluate implementiert.
- Early Stopping zur Vermeidung von Overfitting eingebaut.
- Verschiedene neuronale Netzarchitekturen definiert:
- kleines Modell [64, 32]
- größeres Modell [128, 64, 32]
- Verschiedene Regularisierungsmethoden getestet:
- kein Regularizer
- L1 Regularisierung
- L2 Regularisierung
- L1 + L2 Regularisierung
- Dropout als zusätzliche Regularisierung implementiert.
- Mehrere Modelle automatisiert in einer Schleife trainiert.
- Trainingsverlauf (Loss und Validation Loss) gespeichert.
- Ergebnisse der Modelle in einem Dictionary gespeichert.
- Ergebnisse als Tabelle (DataFrame) zusammengefasst.
- Bestes Modell anhand des Testfehlers ausgewählt.
- Trainingsverlauf des besten Modells visualisiert.

Schwierigkeiten / offene Fragen:

- Kernel-Neustart in VS Code Notebook versucht.
- Fehler durch nicht definierte Variablen (x_train, X_train_scaled) untersucht.
- Unterschiedliche Variablennamen zwischen Codeblöcken korrigiert.
- Notebook-Zellen in korrekter Reihenfolge ausgeführt.

Eintrag 7

Datum: 12.03.2026 10:00-20:00

Titel: Erweiterung und Bereinigung des neuronalen Netzes (Regularisierung L1–L3)

Aus Gruppentreffen: Gruppentreffen 4

Arbeitsschritte:

- Bestehendes Notebook zum neuronalen Netz für die Regressionsaufgabe weiterentwickelt.
- Verschiedene Regularisierungsmethoden (L1, L2, L1+L2 sowie eigener L3-Regularizer) in das Modell integriert.
- Eine allgemeine Modellfunktion (`build_model_regularized`) erstellt, um unterschiedliche Hidden-Layer-Strukturen und Regularisierungen flexibel testen zu können.
- Mehrere Modellkonfigurationen über ein `model_configs` Dictionary definiert (Plain NN, L1, L2, L1+L2, L3).
- Trainingsschleife implementiert, die automatisch alle Modellvarianten trainiert und auswertet.
- Ergebnisse (Test MSE, Test MAE, Parameteranzahl, Trainingsdauer) in einer gemeinsamen Tabelle gespeichert.
- Fehler im Code behoben, insbesondere das Überschreiben der Dictionaries für Ergebnisse und Trainingshistorien.
- Zusätzlich die trainierten Modelle in `trained_models` gespeichert, um später automatisch das beste Modell auswählen zu können.
- Automatische Auswahl des besten Modells über die sortierte Ergebnistabelle (`results_df`) umgesetzt.
- Trainingsverlauf des besten Modells visualisiert.
- Vorbereitung des Vergleichs zwischen neuronalen Netzen und linearer Regression.

Entscheidungen:

- Eine zentrale Modellfunktion für alle Regularisierungsvarianten verwendet, um redundanten Code zu vermeiden.
- Die Modellkonfigurationen über ein Dictionary gesteuert, damit neue Varianten leicht ergänzt werden können.
- Das beste Modell automatisch anhand des Testfehlers bestimmen lassen, statt es manuell auszuwählen.

Schwierigkeiten / offene Fragen:

- Syntaxfehler im Modellkonfigurationsblock (fehlende Kommata) führten zunächst zu Problemen beim Ausführen der Trainingsschleife.
- Zwischenzeitlich wurden Ergebnisse versehentlich überschrieben, da Dictionaries innerhalb der Schleife neu initialisiert wurden.

- Teilweise existieren im Notebook noch ältere Testblöcke für einzelne Modelle, die eventuell später bereinigt oder klarer als Zwischenschritte gekennzeichnet werden sollten.

Eintrag 8

Datum: 13.03.2026 9:00-19:00

Titel: Ausführung und Auswertung des neuronalen Netzvergleichs

Aus Gruppentreffen: Gruppentreffen 4

Arbeitsschritte:

- Notebook vollständig neu gestartet und alle Zellen erneut ausgeführt, um sicherzustellen, dass alle Modelle konsistent trainiert werden.
- Implementierten Modellvergleich mit verschiedenen Regularisierungsmethoden (Plain, L1, L2, L1+L2 und L3) getestet.
- Trainingsschleife überprüft und kontrolliert, ob alle Modellvarianten korrekt trainiert und in den Ergebnisstrukturen gespeichert werden.
- Ergebnisse in einer Tabelle (`results_df`) zusammengeführt und nach Testfehler sortiert.
- Automatische Auswahl des besten Modells anhand des niedrigsten Testfehlers umgesetzt.
- Trainingsverlauf des besten Modells visualisiert und auf mögliche Overfitting-Anzeichen überprüft.
- Vergleich zwischen neuronalen Netzen und linearer Regression vorbereitet.
- Notebookstruktur überprüft und alte Testblöcke als Zwischenschritte identifiziert.

Entscheidungen:

- Das beste neuronale Netz wird automatisch über die sortierte Ergebnistabelle bestimmt.
- Für die Modellarchitektur und Regularisierung wird ein gemeinsames Konfigurationssystem verwendet, um zukünftige Erweiterungen zu erleichtern.
- Die verschiedenen Modellvarianten werden systematisch verglichen, um den Einfluss der Regularisierung auf die Generalisierungsfähigkeit zu untersuchen.

Schwierigkeiten / offene Fragen:

- Sicherstellen, dass ältere Testmodelle im Notebook klar von der finalen Vergleichsstruktur getrennt sind.
- Überlegen, ob einige der früheren Testabschnitte im Notebook für die finale Abgabe reduziert oder stärker kommentiert werden sollten.

Eintrag 9 10:00 - 17:00 Uhr

Datum: 14.03.2026

Titel: Fehlerbehebung und Abschluss Notebook Neuronales Netz

Aus Gruppentreffen: Eigenständig

Arbeitsschritte:

- Fehler im Notebook „05_Neural_Network_Kolja.ipynb“ analysiert und behoben.
- Funktionsaufrufe (`build_model` und `train_and_evaluate`) auf die konfliktfreie Version (`build_model_baseline` , `train_and_evaluate_baseline`) angepasst.
- Modelle mit verschiedenen Architekturen (tieferes und breiteres Netz) erfolgreich ausgeführt und Ergebnisse überprüft.
- Vergleich verschiedener Aktivierungsfunktionen implementiert.
- Lernkurven der Modelle visualisiert und als Grafik gespeichert.
- Vergleich zwischen neuronalen Netzen und linearer Regression erstellt und grafisch dargestellt.
- Ergebnisse in einer Tabelle zusammengeführt und finale Zusammenfassung vorbereitet.

Entscheidungen:

- Das regulärisierte neuronale Netz mit Dropout wird als bestes Modell für den Vergleich mit der linearen Regression verwendet.
- Für den Modellvergleich werden Test-MSE und Test-MAE als Hauptmetriken genutzt.

Schwierigkeiten / offene Fragen:

- Mehrere Fehler durch umbenannte Funktionen im Notebook mussten im gesamten Code angepasst werden.
- Kurzzeitige Probleme beim Neustart des Jupyter-Kernels in VS Code.

Eintrag template

Datum: [Datum]

Titel: [Titel]

Aus Gruppentreffen: [Treffen X oder "Eigenständig"]

Arbeitsschritte:

Entscheidungen:

Schwierigkeiten / offene Fragen: